Bit Manipulation

10 → Decimal $(10)_{10}$ where $()_{10}$ is sort of
↳ base10 Sleep

2	14	0
2	7	1
2	3	1
	1	

⇒ $(1110)_2$ is binary for 14

$(1011)_2$

- $1 \times 2^0 \Rightarrow 1$
- $1 \times 2^1 = 2$
- $0 \times 2^2 = 0$
- $1 \times 2^3 = 8$

⇒ $1+2+8 \Rightarrow (11)_{10}$

Operators

a=5 b=6

AND → a&b → 5&6 → 101 & 110 → $(100)_2$ → $(4)_{10}$

OR → a | b → 5|6 → 101 | 110 → $(111)_2$ → $(7)_{10}$

XOR → a ^ b → 5^6 → 101^110 → $(011)_2$ → $(3)_{10}$

Complemental → !a → !5 → !101 → $(010)_2$ → $(2)_{10}$

Right Shift → a>>1 → 5>>1 → 0101>>1 → $(0010)_2$ → $(2)_{10}$

Left Shift → a<<1 → 5<<1 → 101<<1 → $(1010)_2$ → $(10)_{10}$

- XOR of same numbers is always 0.
- a>>b = a/2^b
- a<<b = a×2^b

Q Swap 2 Nos using Bit Manipulation

a=5 b=7

a = a^b → a = 5^7 b=7

b = a^b → b = 5^7 ⇒ a = 5^7 ⇒ b = 5 a = 5^7

a = a^b → a = 5^7 ⇒ b = 5 ⇒ a = 7 b = 5

Q For n. Calculate the XOR of 1→N in O(1)

N=1 → 1^0 = 1
N=2 → 2^1 = 3
N=3 → 3^3 = 0
N=4 → 4^0 = 4

if (n%4 == 0) return n;
if (n%4 == 1) return 1;
if (n%4 == 2) return n+1;
if (n%4 == 3) return 0;

N=5 → 5^4 = 1
N=6 → 6^1 = 7
N=7 → 7^7 = 0
N=8 → 8^0 = 8

Q Find XOR of numbers b/w range L to R in O(1)

Eg: L=2 R=6 → 2^3^4^5^6

Sol: use the above logic to find XOR(L) and XOR(R)
take XOR of these 2.

we want → 2^3^4^5^6

we have → 1^2^3^4^5^6 (A) and 1^2 (B)

to eliminate (B) from (A) we take their XOR.

⇒ Return XOR(L-1) ^ XOR(R)

ODD/EVEN

if (n&1 == 0) → Even
else → odd

} faster than Modulo

Mask to check ith bit

given n and i, return true if ith bit is set or not

n = 1+4+8+32 ⇒ 45
i = 2

```
  5 4 3 2 1 0
  0 1 0 1 1 0 1
& 0 0 0 1 0 0
  0 0 0 0 0 0
```

if (n & mask == 0) return true
else return false.

- How create a mask? (1<<i)

- Unsetting the last set bit

n = 13 → $(1101)_2$ → last set bit

n = 1101
n-1 = 12 = 1100 (n&n-1) → to remove the last set bit
⇒ $(1100)_2$

for n=12 ⇒ 1100
n-1 = 11 ⇒ 1011
} (n&(n-1))
⇒ $(1000)_2$ → $(8)_{10}$

* if n&(n-1) == 0, n is a power of 2.

Count the no of set bits

```
while (n != 0) {
    if (n & 1 == 1) {
        count++;
    }
    n = n >> 1;
}
return count;
```

TC ⇒ O(MSB)

Method 2

```
while (n != 0) {
    n = n & n-1;
    count++;
}
return count;
```

TC = O(set bits)
↳ bit faster
but same in worst case

$1100\overset{\times}{1} \rightarrow (25)_{10}$
1 1000
1 0000
0 0000